# External Research — Why Quality Systems Fail Despite Green Signals

| Field | Value |
|---|---|
| Revision | 2 |
| Created | 2026-07-22 |
| Last modified | 2026-07-22T15:03:22Z |
| Status | COMPLETE — 7 research rounds + synthesis (76 sources) |
| Scope | Universal (constitution layer) — no project-specific literals |
| Companion | `SOURCES.md` (full citation ledger), `ROOT_CAUSE_ANALYSIS.md` (internal forensics, sibling effort) |

## . Purpose and method

This document cross-references the external evidence base — peer-reviewed empirical software-engineering literature, safety-critical standards, mature open-source project practice, and practitioner post-mortems — against seven measured failure modes observed in a consuming project:

1. **FM-1 — ~52% reopen-per-fix rate** (undercounted: recurrences minted new ids).
2. **FM-2 — 95% of done-claiming items had no regression guard**; 6/6 operator-sampled "done" items broken.

3. **FM-3 — Verification-absence indistinguishable from verification-success** (0 PASS / 5 FAIL / 56 PENDING → exit 0 "not blocked"; release gate never consulted the suite).
4. **FM-4 — Green tests on broken features** ("tests pass but the feature doesn't work").
5. **FM-5 — Measuring instruments produced confident wrong answers** (carrier/false-match + false-null; ten traps in one session).
6. **FM-6 — Source-green treated as done** (artifact / runtime-on-clean-target / user-visible layers unverified).
7. **FM-7 — ~15 tracked items mapping to ~9 root causes** (duplicate/forked fixes on one cause).

Method: multiple aimed research rounds; each source is tagged **CONFIRMS**, **CONTRADICTS**, or **NUANCES** relative to the failure modes above, with the concrete mechanism it suggests. Evidence class is marked: `[PR]` peer-reviewed / measured, `[IND]` industrial report with data, `[DOC]` standard / official project documentation, `[ANEC]` practitioner anecdote / opinion. Per §11.4.6: where the literature disagrees or is silent, that is stated explicitly. All citations carry URL + access date (§11.4.99) in `SOURCES.md`.[1]

---

# Round — Empirical baselines: reopen rates, coverage-vs-effectiveness, mutation testing, flakiness, cost-of-defect[5][2]

## R . Bug-reopen rates: the published baseline is - %; ours is ~ %

`[PR]` The empirical reopen literature is consistent across ecosystems:

- An empirical analysis of reopened bugs across open-source projects (EASE 2016) reports **6%–10% of bugs are eventually reopened**, and that >93% of reopened bugs seriously affect normal operation of the system (S1, S2).

- The Eclipse case study (WCRE 2010) and Zimmermann et al.'s Microsoft Windows study (ICSE 2012) built predictive models over reopen causes; top predictors are the **component**, the **time needed to produce the first fix**, and **who reported the bug** — defects reported by end users (rather than found by review or static analysis) are significantly more likely to be reopened because they are harder to reproduce (S3, S4).
- Reopened bugs take roughly **2.5× longer to finally resolve** (371.4 vs 149.3 days in the EASE 2016 data set) (S1). A 2022 replication in *Empirical Software Engineering* (''Revisiting reopened bugs in open source software systems'') confirms the phenomenon and predictors at larger scale (S5).

**Cross-reference (FM-1): CONFIRMS the severity, CONTRADICTS any normalisation.** A ~52% reopen-per-fix rate is **5–8× the worst published open-source baseline**. The literature treats ~10% as the *pathological* end. Two mechanisms follow directly:

1. The literature's strongest predictors (hard-to-reproduce, user-reported, quick first fix) describe exactly the "fix confirmed at source-green without reproduction" pattern — reopen probability concentrates where the fix was never demonstrated against the reproducing conditions. This independently supports a reproduce-first / verify-on-the-failing-conditions discipline (§11.4.115/ §11.4.146-class).
2. Zimmermann et al. note that **process factors dominate people factors** in reopen prediction — reopens are a *system* property, not a competence property. That matches the internal finding that the same defect class recurred across many hands.

**Honest boundary (§11.4.6):** the published rates count *explicit reopen events on the same id*. Our 52% figure is reopen-per-*fix* and is itself an undercount (recurrences minted new ids). The comparison is therefore conservative in our disfavour — the true gap is larger, not smaller. No published study we found reports a reopen rate anywhere near 50%; the literature has **no answer** for systems operating at that level other than the generic finding that reopens correlate with unverified fixes.

## R . Coverage is not strongly correlated with test-suite effectiveness

[PR] Inozemtseva & Holmes, ICSE 2014 ("Coverage Is Not Strongly Correlated with Test Suite Effectiveness") — 31,000 generated suites over five systems up to 724 KLOC; statement/decision/modified-condition coverage measured against mutation-based fault detection. Finding: **low-to-moderate correlation between coverage and effectiveness once suite size is controlled for; stronger coverage criteria do not help**. The paper received the ICSE most-influential-paper award ten years later (2024), i.e. the field regards it as settled (S6, S7).

**Cross-reference (FM-2, FM-4): CONFIRMS.** "The code was executed" is a different fact from "the behaviour was checked." A suite can be green at high coverage while asserting almost nothing — the mechanism behind green-tests-on-broken-features. The corollary the paper's authors draw is the operative one: coverage identifies *under-tested* code but must not be used as a *quality target*. Any gate[1][3] that counts executed lines (or counts tests, or counts PASSes) rather than *demonstrated fault-detection* inherits this gap.

## R . Mutation testing: the only widely-validated measure of whether a test can fail

[PR] Petrović, Ivanković, Fraser & Just — "Does mutation testing improve testing practices?" (ICSE 2021) and "Practical Mutation Testing at Scale: A view from Google" (TSE 2022). Google runs mutation testing **diff-incrementally during code review** (not over the whole 2-GLOC codebase), with mutant filtering and operator selection based on historical usefulness, across 24,000+ developers / 1,000+ projects (S8, S9). Key practical findings:

- Whole-codebase mutation does not scale and is not necessary; **mutating the changed lines at review time** is the workable form.
- Unfiltered mutants overwhelm developers ("mutant fatigue"); filtering by relevance is what made the programme survive.
- Exposure to mutants measurably changes developer testing behaviour over time (developers pre-empt the mutants).

**Cross-reference (FM-2, FM-4): CONFIRMS the paired-mutation design and sharpens it.** A test that has never been observed to fail is unvalidated instrumentation — the mutation kill is the *only* scalable positive evidence a guard

can detect the defect class it claims. Google's experience adds two cautions our regime should import: (a) mutation effort belongs at the **diff**, not the corpus (the whole-corpus form collapses under its own cost); (b) mutant **relevance filtering** is load-bearing[1][4] — a mutation programme that spams trivial/equivalent mutants gets ignored, which is the alarm-fatigue failure of R1.4 in another costume.

## R . Flaky tests: measured prevalence, taxonomy, and the trust-erosion mechanism

`[PR]/[IND]` Luo, Hariri, Eloussi & Marinov (FSE 2014) — the foundational taxonomy: **async-wait 45%, concurrency 20%, test-order dependency 12%** of flaky causes (S10). Google reports **~16% of tests exhibit flakiness** ("1 in 7 tests written by our engineers occasionally fail in a way not caused by changes to code or tests") and has dedicated infrastructure to quarantine them; ~13% of failed CI builds in OSS studies are flakiness-caused; GitHub (2020) measured ~1 in 11 commits with a red build caused by a flake (S11, S12, S13).

**Cross-reference (FM-3, FM-5): CONFIRMS via the trust mechanism.** The measured harm of flakiness in every industrial report is not the wasted re-run — it is that **engineers stop believing red**. Once red is disbelieved, a gate's output is noise and people route around it (auto-retry, force-merge, "known flaky, ignore"). This is the same signal-integrity collapse as FM-3 (verification absence read as success) arrived at from the opposite direction: FM-3 makes *absence* look green; flakiness makes *red* look ignorable. Both end in gates nobody consults — which is precisely the measured state (release tooling consulting the guard suite zero times). The literature's remedies converge: quarantine flaky tests *visibly* (never silently[1][5]), track them as first-class work, and keep the blocking suite's red rate near zero so red stays believable.

## R . Cost-of-defect-by-stage: directionally supported, magnitudes are folklore

`[PR-critique]` The canonical "1×–10×–100×–1000× by stage" curve (Boehm; repeated via Pressman 1987 citing IBM Systems Sciences Institute "course notes") **does not survive source-tracing**. Bossavit (*The Leprechauns of Software Engineering*) shows the cited studies are misrepresented or untraceable; one cited study found a 2:1 ratio in the *opposite* direction; The Register's 2021 investigation

could not locate the underlying IBM study at all (S14, S15). Directional support (later is costlier) exists — NIST 2002 and Capers Jones' project base — but the specific multipliers are unsupported.

**Cross-reference: CONTRADICTS a common justification pattern — flagged prominently per the brief.** Any internal document citing "100× in production" as measured fact commits the same class of error as FM-5 (confident numbers without a traceable instrument). The *defensible* claims are: (a) defects found by users cost more than defects found by the author (supported, magnitude context-dependent); (b) reopened defects take ~2.5× longer to resolve (R1.1, actually measured). Use those, not the folklore curve. The deeper lesson for a governance corpus: **a rule justified by an untraceable number teaches readers that the corpus's numbers are decorative** — which corrodes exactly the evidence-discipline the corpus is trying to enforce.

## R . Escape-rate and change-failure baselines

`[IND]` Industry benchmarks (DevStats/Plandek metric guides; Capers Jones' DRE data; DORA reports): typical defect-removal efficiency ~85% (≈15% escape), elite escaped- defect rates <10%, DORA elite change-failure-rate band 0–15%, with elite teams achieving both high deploy frequency *and* low failure rates (quality gates built in, not bolted on) (S16, S17). These are vendor-aggregated rather than peer-reviewed, but consistent across independent sources.

**Cross-reference (FM-1, FM-2): CONFIRMS the magnitude statement.** Where measured baselines exist, the observed numbers sit far outside every published band, and the DORA finding removes the speed-vs-quality excuse: elite cohorts are fast *because* their gates are trustworthy, not despite them.

---

# Round — Why green suites miss real defects: oracles, assertions, metamorphic and property-based testing, fault injection

## R . The oracle problem: "did it run" vs "was it right" is the field's named central difficulty

[PR] Barr, Harman, McMinn, Shahbaz & Yoo, "The Oracle Problem in Software Testing: A Survey", IEEE TSE 41(5), 2015 (S18). The *oracle problem* — given an input, distinguishing desired from undesired behaviour — is identified as **the bottleneck of test automation**: inputs can be generated automatically at scale, but deciding correctness cannot, absent an explicit oracle (specification, model, contract, metamorphic relation, or golden reference). Tests without meaningful oracles reduce to crash-detection: they pass whenever the code doesn't throw.

**Cross-reference (FM-4): CONFIRMS and names the mechanism.** "Green tests on broken features" is the oracle problem verbatim: the suite executed the code path (input side automated) but the oracle asserted the wrong layer — file-exists instead of content-correct, exit-0 instead of user-visible-effect, metadata instead of media. The survey's taxonomy also legitimises the derived-oracle strategies already in the governance corpus (sink-side probes, metamorphic relations, golden references) as the *standard* answers, not exotica. Key transfer: an oracle is a designed artifact with its own defect modes — which is why analyzer self-validation (golden-good/golden-bad) appears in the literature as "oracle assessment."

## R . Assertions — not coverage — are what correlates with fault detection

[PR] Zhang & Mesbah, "Assertions Are Strongly Correlated with Test Suite Effectiveness", ESEC/FSE 2015 (S19): across large suites, **assertion quantity and assertion coverage correlate strongly with mutation-detection effectiveness — more strongly than statement coverage** (the direct

complement to R1.2). A 2023–2025 literature line on assertion quality (brittle assertions, unused inputs) extends this: what the assertions *touch* matters, not just their count (S20).

**Cross-reference (FM-2, FM-4): CONFIRMS with an actionable inversion.** The pair (R1.2 + R2.2) is the strongest single result pair in the testing literature for our purposes: *execution* metrics don't predict defect detection, *checking* metrics do. A gate that verifies "a test exists / ran / covered the diff" measures the weak predictor; a gate[2][3] that verifies "an assertion checked the user-visible layer and has been observed to fail on the broken artifact" measures the strong one.

## R . Test smells: assertion-free and unexplained-assertion tests are measured, prevalent decay modes

[PR] The test-smell literature (Bavota et al.; QUASOQ 2024 prevalence studies; controlled experiments on Assertion Roulette refactoring) finds smells in the majority of real test classes — Assertion Roulette alone appears in **>50% of test classes** in several corpora, and auto-generated tests (including LLM-generated: ~47% in Defects4J experiments) reproduce the same smells (S21, S22, S23). Measured impact is on comprehension and maintainability — i.e. on whether a failing test can be *diagnosed*, which feeds the R1.4 trust-erosion loop.

**Cross-reference (FM-4): CONFIRMS, with a caution.** One study found "best" test methods also contain multiple undocumented asserts (S21) — the smell metric is a proxy, not an oracle-strength measure; a gate built naively on smell counts would be another proxy-signal instrument (FM-5 shape). The robust reading: smells locate *candidates*[2][4] for oracle-weakness review; mutation kill (R1.3) remains the ground truth.

## R . Metamorphic testing: the proven oracle-substitute when no golden answer exists

[PR] Compiler-testing programmes built on metamorphic relations (EMI: Orion, Athena, Hermes; C4 for C11 atomics; CTDip) have found **>1,000 confirmed bugs in GCC/LLVM**, including the hardest class (silent miscompilation), by checking

*relations between runs* (equivalent inputs ⇒ equal outputs) instead of absolute expected outputs (S24, S25). Segura et al.'s survey (TSE 2016) systematises the technique (S26).

**Cross-reference (FM-4): CONFIRMS.** Metamorphic relations are the standing answer where the correct output is unknowable in advance (media pipelines, renderers, cross-display routing): same content on route A vs route B must match; pause ⇒ counter freezes; 2× speed ⇒ ~2× frame advance. The compiler evidence shows[2][5] this is not a weaker fallback — it catches the *silent-wrong-output*[5][0] class that assertion-per-case testing structurally misses, which is exactly the FM-4 class.

## R . Property-based testing: measured ~ × mutant-kill rate per test vs unit tests

`[PR]` "An Empirical Evaluation of Property-Based Testing in Python" (OOPSLA 2025): across 426 Hypothesis-using programs / 40 projects under mutation analysis, **an average property-based test detects ~50× as many mutants as an average unit test**; notably, over half the mutants were caught with a *single* generated input — the power comes substantially from the property (the oracle), not only from input volume (S27). "Property-Based Testing in Practice" (ICSE 2024) documents industrial adoption (Amazon, Volvo, Stripe) and the Quviq QuickCheck industrial record (S28).

**Cross-reference (FM-2, FM-4): CONFIRMS R2.2 from a third direction.** Three independent lines (assertion correlation, metamorphic compiler results, PBT mutation kill) converge on one statement: **fault-detection power lives in the oracle[2][6], and example-based green suites are weak oracles**. This is the literature's explanation of FM-4 at mechanism level.

## R . Fault injection / chaos engineering: strong practice consensus, weak formal evidence base

`[IND]/[ANEC]` Netflix's chaos programme (Chaos Monkey → FIT → automated reliability scoring) is the canonical industrial account; vendor and practitioner literature uniformly reports resilience gains ("last affected, first recovered" during AWS outages), and academic work exists on injection realism (system-call-level

error injection) (S29, S30). However — stated per §11.4.6 — **we found no peer-reviewed controlled study quantifying outage reduction attributable to chaos engineering**; the evidence is observational and self-reported.

**Cross-reference (stress/chaos mandates): NUANCES.** The practice is well-supported as a *discovery-pressure* mechanism (it manufactures the failure conditions that example-based suites never exercise — the R4-class "we only see what we test" gap), but claims of measured outage-percentage reduction should not be asserted as established fact. Its strongest defensible framing: chaos injection is metamorphic testing over the availability dimension (the system's user-visible behaviour must be invariant under instance loss), inheriting R2.4's logic rather than its own evidence base.[3]

---

# Round — Verification semantics: "unknown is not a pass", gating that never consults the evidence, and what standards call evidence

## R . The industry's own FM- : GitHub treats skipped required checks as SUCCESS

`[DOC]/[IND]` GitHub's branch-protection semantics: **a required status check that is skipped reports status "Success"** and satisfies the protection rule; required checks must be "successful, skipped, or neutral" to allow merge (S31). Practitioner analysis ("Skippable GitHub Status Checks Aren't Really Required") demonstrates the resulting bypass: any path-filtered / conditionally-triggered required workflow can be skipped — by a commit that avoids the trigger paths — and the PR merges with the check never having run (S32). The ecosystem answer is a third-party enforcement layer ( `wait-for-status-checks` : "require *triggered* checks pass" vs GitHub's naive "require checks pass") (S33).

**Cross-reference (FM-3): CONFIRMS — this is the identical defect in the world's largest CI platform.** The 0-PASS/56-PENDING/exit-0 incident is not a local oddity; it is the default semantics of mainstream tooling. The transferable design points from how the ecosystem patched it:

1. **The gate must enumerate what was *supposed* to run and assert each one actually ran** — the required-set is declared independently of the execution, so absence is detectable (identical to the verdict-coverage rule: `uncovered = registered ∧ topology-present ∧ no-verdict`).
2. **"Success" must be reserved for "ran and passed."** Skip/pending/neutral are distinct outcomes that a *release* seam treats as blocking even where an intermediate seam legitimately tolerates them (seam-dependent semantics — [2][3] matching the rejection of flat "PENDING always blocks").
3. Exit-code aggregation that only sums FAILs re-implements GitHub's bug locally.

## R . Safety standards define "verified" as a property of EVIDENCE, not of activity

`[DOC]` DO-178C (avionics): verification evidence is categorised (review records, analysis records, test records, traceability data, process evidence) and is **sufficient only when it demonstrates the claimed property under the conditions the claim applies to**; artifacts must survive independent review by a non-author, with independence scaling with criticality level (DAL); bidirectional traceability from requirement → test → result is itself controlled lifecycle data (S34, S35). ISO 26262 / IEC 61508 (automotive/industrial) add **confirmation measures** — confirmation review, functional-safety audit, functional-safety assessment — performed with ASIL-proportional independence (S36, S37).

**Cross-reference (FM-2, FM-3, FM-6): CONFIRMS the evidence-chain architecture.** The standards' shared structure is exactly the custody chain: a claim ("done") is invalid without traceable evidence produced under defined conditions and reviewed by a non-author. Two transfers stand out:

- **The existence of evidence does not make it sufficient** (DO-178C's own phrasing) — the standard's counter to metadata-PASS / wrong-layer evidence (FM-4/FM-6).

- **Independence is a graded, costed requirement**, not a slogan — the standards version of author-vs-reviewer separation, and the reason self-certified "done" (FM-2's 6/6-broken sample) is structurally expected in its absence.

**Honest boundary (§11.4.6):** these standards trade enormous cost for this rigor (DO-178C Level A is famously expensive per line). The literature does **not** support importing their full apparatus into fast-iteration development; it supports importing the[3] *semantics*[3] (claims bound to evidence; absence ≠ pass; independence at the highest-risk seams only).

## R . Assurance-case research: confidence comes from eliminated doubt, not accumulated green

`[PR]` Eliminative argumentation (SEI; Goodenough/Weinstock/Klein) formalises assurance confidence as **doubt elimination**: enumerate *defeaters* (reasons the claim could be false — doubts about the claim, the evidence, or the inference linking them) and show each is eliminated; confidence rises only as defeaters fall (S38, S39). A 2025 taxonomy of real-world defeaters and follow-on tooling (CoDefeater, Defeater Cards) finds practitioners systematically miss implicit assumptions, "which can lead to false confidence" (S40).

**Cross-reference (FM-2, FM-5): CONFIRMS at the epistemology layer.** A green dashboard is an *inductive* argument ("we looked N times and saw nothing"); the defeater frame forces the *eliminative* question: "what would make this green wrong, and did we check it?" Every FM-5 instrument trap is a defeater of the evidence class ("the instrument can return silence while blind") that was never enumerated. The control-needle discipline is precisely defeater-elimination applied to measurements — the literature supplies the vocabulary and shows the failure (implicit-assumption blindness) is universal, not local.

---

# Round — How mature projects prevent recurrence: SQLite, Linux, curl, Chromium, and duplicate-tracking baselines

## R . SQLite: the strongest testing regime in open source — and what it still missed

`[DOC]` SQLite's published regime ("How SQLite Is Tested"): ~**100 million lines of test code against ~160 KLOC of product** (≈600:1), 2.4 million test cases per full run, **100% branch and 100% MC/DC coverage** maintained since 3.6.17 (2009) via the proprietary TH3 harness, plus independent harnesses (TCL, SLT), boundary testing, OOM/IO-error/crash injection on every path, and multi-platform runs before release (S41, S42). SQLite explicitly credits full coverage with confidence that "changes made in one part of the code do not have unintended consequences in other parts" — the regression-confidence property.

`[IND]` **The counter-fact, from SQLite's own documentation:** in 2015 AFL fuzzing found **22 crashing inputs in SQLite despite the 100% MC/DC regime**, and SQLite's testing page now states plainly that "code tested to 100% MC/DC will tend to be more vulnerable to problems found by fuzzing" — the two pressures find *disjoint* defect classes (S43, S41). SQLite responded by making fuzzing (including OSS-Fuzz and its own dbsqlfuzz) a permanent pillar alongside coverage.

**Cross-reference (FM-2, FM-4): CONFIRMS both halves of the thesis — flagged as the round's most instructive CONTRADICTS-shaped result.** The world's most coverage-disciplined project *still* shipped fuzz-findable defects: even perfect execution coverage cannot see input-space and state-space classes its oracles never pose. The mature response was not "more of the same metric" but **adding an orthogonal discovery pressure** and keeping both permanently. For a project at the *opposite* end (FM-2: 95% unguarded), the transfer is directional, not literal: the 600:1 ratio is unaffordable and unnecessary; what transfers is (a) regression confidence is *bought* with independent harnesses exercising the same claims, and (b) any single verification lens — including a perfect one — has a complement that only a different lens sees.

## R . Linux kernel: "we don't cause regressions" as an enforced, tracked, personal obligation

`[DOC]` The kernel's documented "first rule": **regressions are forbidden; if one lands, the developer who caused it is expected to fix it quickly; exceptions are extremely rare** ("developers almost always turned out to be wrong when they assumed a particular situation warranted an exception") (S44). Enforcement is social + tooling: a dedicated regressions tracker (regzbot, a full-time tracked mailing-list bot, now under KernelCI), whose state **Linus consults when deciding whether to release or extend the cycle** (S45, S46). A user-visible behaviour break is a regression even if the old behaviour was "wrong" — the reference point is what users experienced, not what the code intended.

**Cross-reference (FM-1, FM-3): CONFIRMS two mechanisms.** (1) The release decision *mechanically consults the regression ledger* — the exact seam that was missing in FM-3 (release tooling consulting the guard suite zero times). (2) Recurrence prevention is attached to *identity and obligation* (the causer fixes it, tracked by id until closed), not to a generic backlog — reopens cannot silently become new ids because the tracker's unit is the regression event bound to the causing commit. Notably the kernel achieves this **without** per-fix regression tests as[4] a[3] hard rule — the binding is bisect + tracked obligation; the test-per-fix discipline below is the complementary form.

## R . curl: per-change verification mass as the recurrence barrier

`[DOC]/[IND]` curl runs **>1,900 regression test cases across ~130 CI environments on every commit and PR (~140,000 test executions per change)**; Stenberg's testing write-ups describe the deliberate accumulation of a test per fixed behaviour and CI-gating everything (S47). The suite is the institutional memory: a fix's test outlives the fix's author.

**Cross-reference (FM-2): CONFIRMS the regression-guard-per-defect policy at practitioner scale** — a mid-sized project (one maintainer, ~180 KLOC) sustains it, demonstrating the policy is not a hyperscaler luxury. The measured contrast: curl's public CVE/bug data shows defects recur rarely once tested; our FM-1 measured the inverse under the no-guard regime.

## R . Chromium/Google: flake management as dedicated, ranked, policy-driven infrastructure

[DOC] Chromium operates Flake Portal / LUCI Analysis: flakes are **detected centrally, clustered, ranked by measured CQ/CI impact, auto-filed with policy-driven bug comments**, and sheriffs may disable a test only through a tracked path (the bug stays open; disabled ≠ resolved) (S48, S49). The sheriffing docs codify the triage ladder: revert culprit CL first; disable only when no culprit can be isolated, and then visibly (S50).

**Cross-reference (FM-3): CONFIRMS the "quarantine visibly, never silently" design.** The load-bearing property is that a disabled/flaky test **remains a tracked liability with an owner and a measured impact score** — it can never quietly become[4][5] "absence read as success."[7] This is the operational form of R3.1's lesson at suite scale.

## R . Duplicate/forked tracking: the published baseline for FM-

[PR] Duplicate-report studies across Mozilla/Eclipse/OpenOffice/NetBeans measure **10–30% duplicate rates** (Mozilla up to ~30%, Eclipse ~17–20%; 2017–2022 replication: 6.6–20.5%), with up to 23% of true duplicates being *textually dissimilar* — keying on subject text alone systematically misses them (S51, S52). The field's response is ranking/classification-based duplicate detection at intake, not post-hoc cleanup.

**Cross-reference (FM-7): CONFIRMS with a caveat.** ~15 items over ~9 root causes is a ~40% duplication rate — above the worst published tracker baseline, but the baselines count *reporter-side* duplicates at intake; FM-7's forked-fix phenomenon (multiple *fixes* engineered for one cause) is a more expensive downstream form the duplicate literature does not directly measure (**no direct literature answer found** for fix-level forking rates; §11.4.6). The textual-dissimilarity finding does transfer: intake dedup keyed on normalised (subject, scope) — not bare substring — matches the measured failure mode of naive matching.

---

# Round — Instrument reliability: why measuring tools return confident wrong answers, and what disciplines exist against it

*(This is the least-covered area in mainstream SE literature relative to its observed impact — stated up front per §11.4.6. The strongest material comes from three adjacent[5] fields[1]: shell-language empirical studies, static-analysis adoption research, and experimental-science control methodology.)*

## R . Shell scripting is a measured minefield of silent-wrong-answer classes

`[PR]` Dong et al., "Bash in the Wild: Language Usage, Code Smells, and Bugs" (TOSEM 2022) — >1M GitHub shell scripts: the dominant defect sources are **quoting/word- splitting and error-handling**, with a recurring bug theme of "chains of commands where developers assumed the success of each command" (S53). `[DOC]` The practitioner canon (Wooledge BashFAQ/105; Oil shell's error-handling analysis) documents that `set -e` /"unofficial strict mode" has **known holes and false triggers** (command substitution, conditionals, `local` masking, SIGPIPE-under-`pipefail` when a consumer like `head` exits early), such that even the community's own hardening idiom produces both silent continuation *and* spurious abort (S54, S55). The `ps | grep` self-match is old enough to have a canonical-workarounds literature ( `grep -v grep` , the `[c]lass` trick, `pgrep` — which itself matches pattern carriers) (S56).

**Cross-reference (FM-5): CONFIRMS every observed trap class as a *known, named, recurrent* hazard — not project-specific bad luck.** Three structural facts from this literature matter:

1. **The failure mode of most shell instruments is a clean, plausible, wrong answer** — empty output, exit 0, or a carrier match — not a crash. Parse-checking ( `sh -n` /ShellCheck) operates at parse time and cannot see any of it; the TOSEM study's "assumed success of each command" theme is exactly the pipeline-exit-status trap.

2. **Exit-status semantics are per-tool conventions, not a system contract**: `grep` exits 1 on no-match (meaningful), `find` exits 0 on no-match (silent), pipelines return the last command's status unless `pipefail` — and `pipefail` introduces its own false positives. Any aggregator over these without per-tool handling is guessing.
3. The community's best answer is **defense-in-depth plus explicit per-command error handling**, not a strict-mode flag — i.e. there is no configuration that makes shell instruments trustworthy without per-measurement validation.[5][2]

## R . Static-analysis adoption research: false positives destroy tool trust at measurable thresholds

`[PR]` Johnson, Song, Murphy-Hill & Bowdidge (ICSE 2013): interviews show false positives and warning presentation are the primary reasons developers abandon static-analysis tools despite believing in them (S57). Sadowski et al., "Lessons from Building Static Analysis Tools at Google" (CACM 2018): Google's operational doctrine — **the *perceived/effective* false-positive rate is decided by developers, not tool authors; code-review-blocking checks must stay under ~10% effective false positives**; their earlier FindBugs deployments failed on "warning blindness," and Tricorder succeeded by curating checks and deleting noisy ones (S58, S59).

**Cross-reference (FM-3, FM-5): CONFIRMS the two-sided bluff economics.** A guard that false-refuses (FM-5's carrier-matching build guard; the "swap full" pre-flight) burns trust from the same budget as a missed defect: past a modest noise threshold, humans *and agents* route around the instrument, after which its true positives are also lost — the mechanism by which FM-5 feeds FM-3. Google's remedy set transfers directly: measure each check's effective FP rate, give every refusal an actionable evidence payload, and **retire or fix any check exceeding the noise budget** rather than letting it teach consumers to ignore the fleet.

## R . Experimental science's answer to blind instruments: negative and positive controls

`[PR]` Negative-control methodology (Lipsitch, Tchetgen Tchetgen & Cohen, *Epidemiology* 2010; 2021 selective review; 2023 scoping review): laboratory science *routinely* runs (a) **positive controls** — a stimulus known to produce the effect, proving the instrument can detect it; and (b) **negative controls** — an exposure/outcome known to have no causal link, proving the instrument doesn't manufacture effects; observational sciences imported both to detect confounding, selection and **measurement bias** (S60, S61). The validity condition is explicitly stated in the literature: a control is informative only insofar as it is **subject to the same sources of error as the real measurement** (S62).

**Cross-reference (FM-5): CONFIRMS the control-needle discipline as an instance of century-old experimental method — and sharpens its validity condition.** The control-needle rule ("a null is not evidence until a known-present needle traverses the same path") is a positive control; the golden-FALSE/carrier fixture is a negative control. The epidemiology literature contributes the precise failure mode of naive controls: **a control that does not share the measurement's load-bearing error sources certifies nothing** — the exact reason a bare-literal needle cannot certify a dialect-dependent query (it doesn't share the query's failure surface). This is independent, external confirmation of the needle-class-matching requirement, derived in a field with no connection to shell tooling.

**Honest boundary (§11.4.6):** we found **no SE-specific empirical literature measuring the prevalence of wrong-answer *instrumentation* in test/gate tooling itself** (as opposed to defects in shell scripts generally). The closest analogues are the flaky-test literature (R1.4 — tests as unreliable instruments) and meta-testing/mutation of analyzers (R1.3). The gap is real: the practice of validating one's *measurement path* per query appears in experimental science and in isolated practitioner lore, but not as a named SE discipline.

---

# Round — The process/human layer: why documented rules fail to bind

## R . Checklists: the effect is real where implementation is deep — and NULL where the artifact is merely adopted

`[PR]` The two canonical positives: the Keystone ICU project (Pronovost) nearly eliminated central-line infections across Michigan ICUs; the WHO Surgical Safety Checklist study (Gawande et al., NEJM 2009, 8 hospitals) roughly halved deaths (1.5%→0.8%) and cut complications (11%→7%) (S63, S64). **The critical replication:** Urbach et al., NEJM 2014 — after Ontario *mandated* the checklist across 101 hospitals (~110k procedures before/after), **no statistically significant improvement in any outcome** (deaths 0.71%→0.65%, p=0.13) (S65). The reconciliation accepted in the patient-safety literature (AHRQ PSNet): the positives were implementation programmes (training, culture change, measurement, local champions) in which the checklist was the visible tip; the null was **artifact adoption without the programme** — mandated paper compliance (S66).

**Cross-reference (rule-corpus growth; FM-2/FM-3): CONTRADICTS the implicit theory that documenting a rule produces the behaviour — the single most important negative result of this research effort.** A governance corpus that responds to each incident by writing another anchor is running the Ontario experiment repeatedly. The literature's discriminator between the positive and null results is *whether the rule is embedded in the work system* (who executes it, when, with what forcing function and what measurement) — in SE terms: a rule bound to a seam (hook, gate, refusing writer) is Keystone; a rule that exists as prose is Ontario[6][2]. This independently derives the "enforced-not-advisory / a rule documented but not hooked catches nothing" doctrine.

## R . Work-as-imagined vs work-as-done: routine noncompliance is the normal state of rich rule systems

`[PR]/[DOC]` Hollnagel's Safety-I/Safety-II white paper and the resilience-engineering literature: a gap between work-as-imagined (procedures) and work-as-done (practice) is **unavoidable**; systems function *because* operators adapt;

**routine violations become invisible to the group committing them** and are tolerated by supervision until an accident makes them legible (S67, S68). The aviation-crew noncompliance literature maps the drivers: conflicting goals, time pressure, and procedures that don't fit the situation (S69).

**Cross-reference (FM-2, FM-6): CONFIRMS-and-reframes.** "95% of done-claims had no guard despite a documented guard mandate" is not anomalous wickedness; it is the textbook drift pattern — the documented process diverged from the practiced process, and *nothing measured the divergence*. The literature's remedy is not "insist harder": it is (a) instrument the gap itself (measure work-as-done: e.g. a sweep that counts guardless done-claims — which is what finally exposed FM-2), and[6] (b) reduce the rule-to-reality distance (rules[3] that fit the actual workflow get followed; rules that require heroics get adapted away).[9][0]

## R . Alert fatigue: measured % override rates — the quantitative ceiling on interruptive gating

`[PR]` Clinical decision support meta-analysis: **~90% (CI 85–95%) of drug-interaction alerts are overridden**; even "very severe" alerts are overridden at ~88%; a large fraction of overrides are *appropriate*, because the alert corpus is mis-calibrated (S70, S71). This is the most heavily-measured human-response-to-automated-gates dataset in any industry.

**Cross-reference (FM-3, FM-5): CONFIRMS R5.2 from an independent field, with a number.** When a gate fleet's precision drops, override becomes the *rational* norm, and the rare true alarm is overridden along with the noise. For a governance system this sets a design constraint: **every added blocking check spends from a finite attention/trust budget**, and past the calibration point additional checks reduce total enforcement. The measured CDS remedies map one-to-one: tier alerts (interruptive only for the highest-severity subset — cf. seam-dependent blocking), suppress known-noise classes, and audit override reasons (an escape-with-rationale trail is the SE equivalent).

### R . Security-policy compliance research: overload produces workarounds, not defiance

[PR] Security-fatigue studies (longitudinal daily-compliance designs; 2025–2026 university studies): cumulative rule burden depletes self-regulation; noncompliance is **typically non-malicious overload response** — employees ignore warnings and build workarounds to stay productive; policy complexity itself predicts unintentional noncompliance; 41% cite bureaucracy/process overload as a primary barrier (S72, S73).

**Cross-reference (rule-corpus-size question): NUANCES — and an honest gap.** The literature robustly supports the *direction* (more + more-complex rules ⇒ more noncompliance via fatigue, with self-regulation depletion as the mechanism), but **we found no study establishing a numeric threshold** ("N rules is past the limit") that would let us evaluate a ~224-anchor corpus against a measured line (§11.4.6: no answer exists in the literature we reached). The defensible engineering conclusions: (a) the marginal anchor has *negative* expected value once fatigue effects dominate, (b) compliance concentrates in whatever subset is mechanically enforced (R6.1), which argues for a small enforced core plus reference material, rather than a uniformly "mandatory" corpus where the effective mandatory subset is chosen by fatigue instead of by design.[7]

---

## Round — Where escaped defects actually live: error paths and untested changes[5][8]

### R . % of catastrophic failures are mishandled *signalled* errors; % catchable by trivial tests

[PR] Yuan et al., "Simple Testing Can Prevent Most Critical Failures" (OSDI 2014) — 198 user-reported production failures across Cassandra, HBase, HDFS, MapReduce, Redis: **92% of catastrophic failures were incorrect handling of non-fatal errors the software itself had explicitly signalled**; in 58%, the fault

was catchable by trivial error-handling tests (empty handlers, "TODO" handlers, over-broad catch-and-abort); 77% of all production failures were reproducible in a unit test; 98% manifest on ≤3 nodes (S74).

**Cross-reference (FM-4, FM-2): CONFIRMS with a precise target.** The green-but-broken pattern concentrates on paths the happy-path suite never enters: the error/ fallback/ recovery arms. The paper's most transferable finding is its *shape*: the system **told itself** something was wrong (the error was signalled) and the handling layer discarded the signal — the runtime twin of FM-3/FM-5 (a signal existed; the consumer converted it to silence). Their tool (Aspirator) found hundreds of real bugs[7][2] by simply flagging empty/trivial exception handlers[5] — cheap static discovery-pressure on exactly the class that kills systems.

## R . Untested *changes* carry ~ × the defect density; about half of changes ship untested

`[PR]/[IND]` Eder, Hauptmann, Junker, Juergens et al., "Did We Test Our Changes?" and the follow-on Test-Gap-Analysis programme (CQSE/Teamscale; 14-month industrial case, ~340 KLOC): **~50% of code changes reached production untested, and untested changes were ~5× more likely to contain defects** than the rest of the system; the derived practice ("test-gap analysis") diffs the release against test-execution coverage and surfaces changed-but-never-executed code before ship (S75, S76).

**Cross-reference (FM-2, FM-6): CONFIRMS the diff-scoped guard policy with an effect size.** This is the quantitative bridge between R1.2 (corpus coverage is a weak signal) and practice: coverage *of the change* — did anything at all execute the new behaviour before release? — is where the defect mass concentrates. It also independently justifies FM-6's layer framing: "source-green" without evidence that the *shipped artifact's changed behaviour* was exercised is precisely the measured 5×-risk population.

---

# Synthesis — cross-referencing all rounds against the seven failure modes

## SYN- The five strongest literature-supported mechanisms for OUR failure modes

**M1 — Bind every "done" to an oracle that has been observed to fail (FM-1, FM-2, FM-4).** Convergent evidence: assertion-effectiveness correlation (R2.2), PBT's ~50× mutant kill living in the property (R2.5), mutation testing as the only scalable proof a test can detect its defect class (R1.3), reopen-predictor concentration on unreproduced fixes (R1.1), and the 5× defect density of untested changes (R7.2). Mechanism: a done-claim is accepted only with a guard that (a) failed on the broken artifact, (b) passed on the fixed artifact, (c) is mutation-paired at the diff. This is the highest-leverage single control the literature supports, and it is *diff-scoped* — Google-scale evidence says whole-corpus forms collapse (R1.3).

**M2 — Make absence a first-class, blocking, *enumerated* state (FM-3).** GitHub's skipped-counts-as-success is the same bug at industry scale, and the ecosystem fix is the same design (R3.1): declare the required set independently of execution; compute `uncovered = required − ran`; release seams block on uncovered ≠ ∅ as hard as on FAIL. DO-178C/ISO 26262 supply the mature semantics (claims are invalid without sufficient evidence; independence at the highest-risk seams) (R3.2); the kernel supplies the proof it scales socially (the release decision mechanically consults the regression ledger) (R4.2).

**M3 — Validate the instrument per measurement: positive + negative controls with shared error surface (FM-5).** Experimental science has run this control regime for a century; its validity condition (the control must share the measurement's error sources) independently derives the needle-class-matching rule (R5.3). Shell-language empirical data confirms the ambient instrument substrate fails *silently and plausibly* by default (R5.1), and no strict-mode configuration substitutes for per-measurement validation. Analyzer self-validation via golden-good/golden-bad pairs is the same mechanism at suite level (R1.3, R2.4).

**M4 — Protect the trust budget: precision-manage every gate, quarantine visibly, report evidence on every refusal (FM-3 ← FM-5).** Measured: ~90% override at mis-calibrated alert fleets (R6.3); <10% effective-FP bar at Google with

deletion of noisy checks (R5.2); flaky-red erosion of suite credibility (R1.4); Chromium's tracked-liability quarantine so silence never impersonates health (R4.4). Mechanism: a small, fast, near-zero-false-positive blocking core; everything else advisory; every block ships its resolved evidence; noisy checks are fixed or retired, never tolerated.

**M5 — Enforce at seams, not in prose; instrument the imagined/done gap (FM-2, FM-6, corpus-size).** The Ontario null result (R6.1) shows rule-artifact adoption without workflow embedding produces no effect; work-as-done drift is the *normal* state (R6.2); fatigue research shows the marginal unenforced mandate has negative expected value (R6.4). Mechanism: a rule either binds at a write-seam (hook/gate/refusing writer, with a tested runtime state) or is explicitly reference material[2]; and the divergence between documented and practiced process is itself a measured, alarmed quantity (the FM-2 sweep generalised).

## SYN- What CONTRADICTED our assumptions (surfaced per the brief)

1. **"Write the rule and the behaviour follows" — contradicted** by the strongest RCT-adjacent evidence in this entire corpus (Ontario checklist null, R6.1) and the compliance-drift literature (R6.2). The corpus's own history (rules restated repeatedly while the measured violation persisted) is the same result locally.
2. **"More blocking checks ⇒ more safety" — contradicted** by alert-fatigue numbers (90% override, R6.3) and Google's static-analysis experience (R5.2): past the calibration point, added interruptive checks *reduce* total enforcement by spending shared trust.
3. **"100× cost-by-stage" as measured fact — contradicted** (R1.5): the canonical curve is untraceable folklore. The defensible, actually-measured local statement is R1.1's ~2.5× resolution-time penalty for reopened defects.
4. **"Perfect coverage would have saved us" — contradicted at the limit** by SQLite (R4.1): 100% MC/DC still shipped 22 fuzz-findable crashers, and SQLite's own docs state coverage-tested code tends to be *more* fuzz-vulnerable. Orthogonal discovery pressure is not optional at any maturity level.
5. **"Strict-mode the scripts and the instruments become trustworthy" — contradicted** by the shell literature itself (R5.1): `set -euo pipefail` has documented holes and its *own* false-positive classes; only per-measurement controls close the gap.

6. **Partially contradicted — "operator-attended checklists are weak":** the checklist evidence cuts both ways; where implementation is deep (Keystone/WHO[3]), human checklists produced some of the largest safety effects ever measured[1][1][4][6] (R6.1). The variable is embedding, not the human.

## SYN- Where the literature had NO answer (§ . . — stated, not manufactured)

1. **No published baseline for reopen rates near 50%** (R1.1) — the pathology band in the literature ends around 10%; comparisons beyond that are extrapolation.
2. **No SE literature on wrong-answer rates of the measurement tooling itself** (grep/pipeline/exit-status instruments used *inside* gates) — the nearest fields are flaky-test research and analyzer mutation; the per-query control-needle discipline has no named SE counterpart we could find (R5.3).
3. **No numeric threshold for governance-corpus size vs compliance** (R6.4) — the direction is established (fatigue → workarounds), the dose-response curve is not.
4. **No measured rate of fix-level forking** (multiple engineered fixes for one root cause, FM-7) — duplicate-report literature measures intake duplicates (10–30%), not downstream duplicate *fixes* (R4.5).
5. **No peer-reviewed quantification of chaos engineering's outage reduction**[4] (R2.6) — practice consensus, observational evidence only.

## SYN- Evidence-class summary

Of the 76 sources in `SOURCES.md` : the load-bearing claims (coverage/assertion correlations, mutation-at-scale, flakiness prevalence, reopen rates, oracle problem, error-handling failure concentration, test-gap 5×, alert-override 90%, checklist positive+null, negative-control methodology) are all `[PR]` peer-reviewed/measured. Industry-benchmark bands (DORA/DRE) and chaos-engineering effectiveness are `[IND]` / `[ANEC]` and are marked as such wherever cited. No conclusion above rests solely on vendor assertion.

## SYN- Cross-map to the internal forensics (`ROOT_CAUSE_ANALYSIS.md`)

The sibling internal analysis derives nine physical causes (PC-1..PC-9) from the measured record. External correspondence, for navigation (no duplication either way):

| Internal cause | External evidence base |
|---|---|
| PC-1 status-without-custody | R3.2 (evidence-sufficiency standards), R4.2 (tracked obligation), M1 |
| PC-2 absence ≡ verification | R3.1 (GitHub skipped=Success), R3.3 (defeaters), M2 |
| PC-3 source-green read as done | R7.2 (untested-change 5×), R3.2 (evidence layers), M1/M2 |
| PC-4 rules as prose, not seams | R6.1 (Ontario null), R6.2 (work-as-done drift), M5 |
| PC-5 instruments produced false findings | R5.1–R5.3 (shell hazards, FP fatigue, controls), M3 |
| PC-6 recurrences fragmented across ids | R1.1 (reopen predictors), R4.5 (dedup baselines + textual dissimilarity) |
| PC-7 guards never run where the defect lives | R4.4 (visible quarantine), R3.1 (required-set enumeration), M2 |
| PC-8 wrong-layer oracles | R2.1 (oracle problem), R2.2/R2.5 (oracle strength), R7.1 (error paths), M1 |
| PC-9 ambiguous done-vocabulary | R3.2 (standards' claim/evidence vocabulary), R6.3 (calibrated severity tiers), M4 |